Holdout

Offers only where they change the outcome.

Bot @navpreet_offers_bot
Dashboard offer-engine-nu.vercel.app
Repo github.com/Navkaur-codes/Offer-Engine

Most discount engines send 20% off to everyone and count every sale that follows as a win. But many of those customers would have bought anyway, so the discount just gave away margin. **Holdout decides, for each customer and moment, whether an offer will actually change what they do, and only then which offer and when.** It sends the offer on Telegram, sends offers **automatically** on a schedule, and gives marketers a dashboard that proves the result. It is named after the group that deliberately gets nothing, which is how we measure what offers really caused.

+₹91,760 extra sales offers caused, vs control groups, last 12 months	**₹14,916** margin saved by not paying people who'd buy anyway	**100%** of offers re-checked by the same rule validator before sending	**~2 s** per AI decision; ~25 ms when the rules decide

Where ₹91,760 and ₹14,916 come from (measured, not counted)

- **Campaign** = one offer sent to many customers at once, with its own coupon. The synthetic year has six (Diwali 2025, Black Friday, Lohri, Holi, Summer sale, Independence Day; `generator.py`, `CAMPAIGNS`); five fall in the last 12 months.
- **Control group** = a random 10% of each campaign's customers who get nothing (`CONTROL_SHARE = 0.10`). Random means the only difference is the offer, so (offer group's buy rate − control's buy rate) × customers × typical basket = sales the offer **caused**. Each campaign is compared only with its own control, then summed.
- **Margin saved**: the control group also shows how many coupon users would have bought anyway; that share of the coupon cost is ₹14,916. **Honest loss**: Lohri's blanket free shipping reached customers who bought less than their control (−₹24,320 profit), which is the case for targeting.
- **Proof from raw data**: the `campaign_history` table on Neon (one row per customer per campaign: control or not, bought or not, revenue) recomputes the same ₹91,760 with one SQL query (in the README). Live offers to real people have no control group, so they don't move it; it is a rolling 12-month window.

How one decision is made

Event	Pre-gate	Policy	AI agent	Validator	Sent + logged

- **Rules set the limits; AI personalises inside them.** Marketers write limits as sentences. A Groq agent (gpt-oss-120b, read-only tools, strict JSON, 20 s) picks and words the offer. **The pre-gate, the agent's own self-check and the final validator all call the same** `evaluate()`, so they can never disagree; a property test pushes thousands of random decisions through it and nothing outside the limits passes.
- **Never silent, never unexplained.** Timeouts, rate limits, invalid JSON, two failed validations or an exhausted AI budget hand over to a deterministic rules fallback. Every decision, including blocked and "no offer", is logged with a plain-English reason built from numbers and rule results, never from the AI's own words.
- **Incrementality first.** Every synthetic customer hides a type (persuadable, sure-thing, lost cause, sleeping dog) the engine must infer. A T-learner uplift model trained on the campaigns predicts each offer's effect; expected incremental profit = uplift × basket × margin − cost, and only offers that pay for themselves are allowed (zero-cost perks first). Layered rules (Global → Festival → Persona → Individual) can only tighten.
- **Culturally safe.** Religion is never inferred from a name; festival greetings go only to customers who opted in and come from the festivals table, never the AI. Consent is immediate (`/pause`, `/stop`); no proactive message 21:00–09:00 IST.

Try it yourself in 5 minutes

Step	Do this	You'll see
1 · Join	Scan the QR (or the Telegram icon in the dashboard's top bar), send `/start`, answer 3 taps	A profile with matching history and a welcome offer as a picture card
2 · Shop	`/simulate cart`, tap **?** Why this?	An offer through the full pipeline, then why you, why this offer, the rules checked
3 · Rules	`/simulate purchase` then `/simulate cart`	No offer: the 48-hour rule blocks it, and the bot says so
4 · Automatic	Dashboard → Automations → **Personal picks** → **Run now** (or wait for 11:00 / 18:00)	A run report per customer; picks in your DM and the Customer Lab group; today's spend and margin kept update

Scan to try
@navpreet_offers_bot

Holdout

Offers only where they change the outcome.

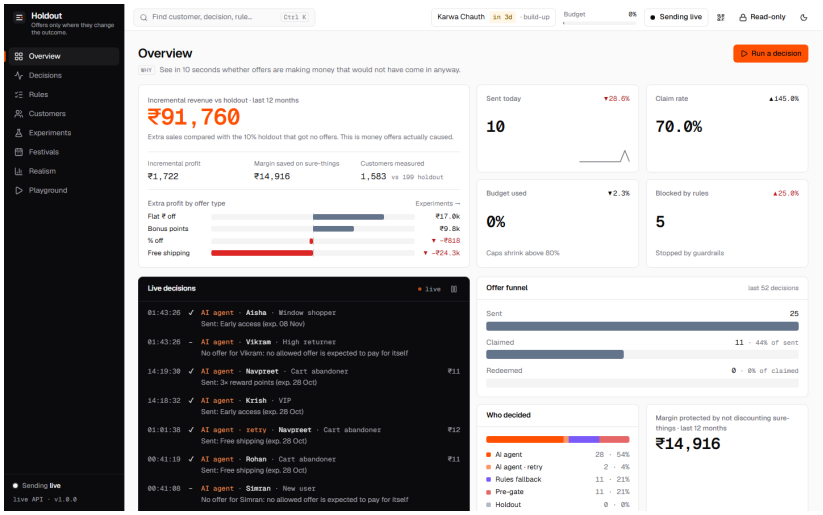
Bot @navpreet_offers_bot
Dashboard offer-engine-nu.vercel.app
Repo github.com/Navkaur-codes/Offer-Engine

What's in it

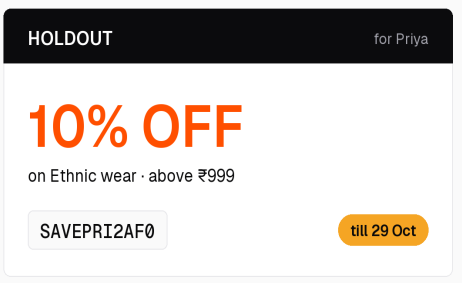
Part	What it does
Telegram bot	`/start`: 3 taps build a profile with synthetic history and a welcome offer. `/simulate cart · browse · purchase · inactive · points_expiry · festival · payday · persona` fire real events through the full pipeline. Picture cards with ✓ Claim · ⏰ Remind me · Not for me · ? Why this? Plus `/why /offers /history /profile /pause /resume /stop`; admin `/summary /killswitch /campaign /impersonate`.
Automation	Abandoned-cart nudges (every 5 min), points-expiry reminders (10:00), **personal picks** for every Telegram and Lab customer from their own profile (11:00 and 18:00, or **Run now** with a per-customer run report and offer mix), a 09:05 flush of offers held during quiet hours, and a 21:00 digest whose numbers are computed in code. Overview shows today's spend and margin kept live.
Customer Lab	A Telegram group where 10 synthetic personas, voiced by "Lab Shopper" bots, shop and react in character; different personas get different offers, and some get nothing on purpose. `/lab date` time-travels to any festival phase.
Dashboard	Overview (one orange hero KPI, live stream, automations) · Decisions with the **Pipeline Lens** (each step ✓/✗) and agent trace · **Rules as sentences** with impact preview, versions and revert · Customers · Experiments · Festivals · Realism · **Playground** with a "force LLM failure" toggle · campaign dry-run · kill switch and AI budget always visible · ⌘K · bot QR.
Synthetic data	~1,500 customers, 12 months simulated day by day: funnels, lognormal baskets, seasonality, payday and festival lifts, churn, returns, messy fields. Benchmarks in range: top 20% ≈ 75% of revenue, cart abandonment ≈ 69%.

Judging criteria → evidence

Criterion	Evidence
Core functionality	Live for anyone: QR → bot → harness → evaluate() → Telegram card → log → dashboard; offers also sent automatically; AI with a guaranteed rules fallback; every decision explained.
Data realism	Day-by-day behaviour simulation; Realism page compares each statistic with a cited benchmark band (✓/✗); campaigns with real control groups in the data.
Usability for marketers	Rules as sentences with editable numbers, impact preview and one-click revert; Run now with a readable report; "why this page exists" on every page.
Design	"Signal" system: cool monochrome, one orange accent, Pipeline Lens, terminal-style stream; no gradients or pie charts; status never by colour alone; Lighthouse accessibility 100.
Advanced functionality	Tool-using LLM agent in a harness; T-learner uplift + holdout; token budget guard (120B → 20B → rules); top-5 agent batching; scheduled personal picks; multi-bot Lab; time-travel.
Business relevance	Measured incremental revenue and margin saved; zero-cost perks first; budget pacing; festival calendar; consent and quiet hours.



Overview: the one orange number is revenue offers actually caused, vs control groups.



Offer card, drawn by the server in the same design system and sent as a photo.

Built with FastAPI · SQLModel · APScheduler · python-telegram-bot · Groq · scikit-learn · Pillow · React · TypeScript · Tailwind · Recharts. Hosted free on Render, Neon Postgres and Vercel. **369 backend tests** (property test on the validator, AI failure cases, every fallback message validated).

How we built it: five Claude Code sessions in parallel worktrees, coordinated by CLAUDE.md (non-negotiables), shared contracts with stubs and a "detective" trap list; an integrator merged them at gates, each with tests, a smoke test and a live check. The human made every commit.